

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

ALTERNATIVE SOLUTION DESIGN

Course Outcome Covered: CO 5

Assessment Tool: Alternative Solution Design | Weightage: 20%

Student Name: K Phani Sridhar Yogi

Register No.: 192311091

QUESTION 1: REST vs SOAP for a Mobile and Web Application

A company needs a web service that will be consumed by both mobile and web applications. The main requirements are fast response time, scalability for a growing user base, simple client development, and secure communication. Two suitable approaches are REST and SOAP.

Comparison of REST and SOAP

Criteria	REST	SOAP	Evaluation
Architecture	Resource-oriented; commonly uses HTTP methods such as GET, POST, PUT and DELETE.	Protocol-based XML messaging with defined envelopes.	REST is simpler for application APIs.
Performance	Usually lightweight, commonly using JSON; less processing overhead.	XML messages are more verbose and require additional processing.	REST is generally faster for mobile/web applications.
Scalability	Stateless requests fit load balancing, caching and horizontal scaling.	Can scale, but messages and enterprise features may add overhead.	REST is preferable for large-scale consumer applications.
Security	HTTPS, OAuth 2.0, JWT and API gateways can provide strong security.	HTTPS plus WS-Security supports message-level security.	Both can be secure; REST is sufficient when designed correctly.
Interoperability	Works naturally with browsers, mobile apps and many programming platforms.	Strong formal standards and contract-based interoperability.	Both are interoperable.

Selected Solution: REST API

I would select REST for this mobile and web application. REST is lightweight, uses standard HTTP operations, and commonly exchanges JSON, which reduces bandwidth and parsing overhead. Its stateless design makes it easy to place multiple application servers behind a load balancer. HTTP caching can also reduce repeated requests and improve response time.

- Performance: JSON payloads and HTTP semantics provide low overhead compared with typical SOAP XML messages.
- Scalability: Stateless REST services can be horizontally scaled by adding application servers.
- Security: HTTPS should be mandatory, while OAuth 2.0/JWT, role-based authorization, rate limiting, input validation, and an API gateway can protect APIs.
- Maintainability: REST resources are straightforward to consume from Android, iOS, browsers, and other web clients.

Alternative: SOAP can be selected when the company requires formal contracts, WS-Security, enterprise transactions, or other WS-* features. However, for a typical mobile/web application where speed, simplicity and scalability are priorities, REST is the better solution.

QUESTION 2: Scalable Service Architecture

An online platform with a large number of simultaneous users should avoid a single server and a single point of failure. A scalable architecture should separate the client, API entry point, application services, caching layer, and database so that each component can be scaled independently.

Proposed Architecture

Client (Web/Mobile) → CDN/WAF → Load Balancer/API Gateway → Multiple Application Servers
→ Cache + Database Cluster

- Client layer: Web browsers and mobile applications send HTTPS requests to the platform.
- CDN/WAF: Static content is delivered close to users, while the web application firewall filters common malicious traffic.
- API Gateway/Load Balancer: Authenticates requests where appropriate, applies rate limits, routes traffic, and distributes requests across healthy application servers.
- Application servers: Multiple stateless instances process business logic. New instances can be added when traffic increases.
- Cache: Frequently requested data is stored in a distributed cache to reduce database load and improve response time.
- Database: A primary database handles writes while read replicas can serve read-heavy workloads. Backups and replication improve availability.
- Monitoring: Logs, metrics, tracing, health checks, and alerts identify failures and performance bottlenecks.

Alternative Design: Microservices with Asynchronous Processing

For a very large platform, the application can be divided into independently scalable services such as User Service, Order Service, Payment Service, and Notification Service. An API gateway routes client requests to the appropriate service. A message queue can handle non-immediate tasks such as emails, notifications, report generation, or background processing.

- Independent scaling: Only heavily used services need additional instances.
- Fault isolation: Failure of one service is less likely to stop the entire platform.
- Asynchronous processing: Queues absorb traffic spikes and prevent slow background tasks from blocking user requests.

- Independent deployment: Services can be updated without rebuilding the entire application.

For a small or medium platform, a modular monolith with a load balancer and cache may be more feasible because it has less operational complexity. For very high traffic and independently changing business domains, the microservices alternative provides greater scalability.

QUESTION 3: Secure Communication for a Banking Application

Banking applications transmit highly sensitive information, including account details, transaction requests, and authentication data. The selected communication protocol must protect confidentiality and integrity, authenticate the server, and provide reliable delivery.

Protocol Analysis

Protocol	Advantages	Limitations
HTTP	Simple and widely supported.	Data is not encrypted; unsuitable for sensitive banking traffic.
HTTPS (HTTP over TLS)	Encrypts traffic, authenticates the server using certificates, and protects message integrity.	Requires correct TLS configuration and certificate management.
FTP	Designed for file transfer.	Not appropriate as the primary protocol for real-time banking APIs.
WebSocket over TLS	Supports secure, persistent, bidirectional communication.	More complex than required for ordinary request/response banking APIs.

Selected Protocol: HTTPS with TLS

HTTPS using modern TLS is the most suitable protocol for banking client-server communication. TLS encrypts data in transit, helps prevent eavesdropping and tampering, and authenticates the server through digital certificates. Banking APIs can use HTTPS together with strong application-layer authentication and authorization.

- Security: Use TLS 1.2 or preferably TLS 1.3 where supported, valid certificates, secure cipher suites, strong authentication, and appropriate session/token controls.
- Reliability: HTTP provides established request/response semantics, while TLS protects message integrity during transmission.
- Performance: TLS introduces some overhead, but modern TLS implementations and connection reuse make HTTPS practical for high-volume APIs.
- Additional protection: Use OAuth 2.0 or another strong authentication mechanism, role/permission checks, transaction signing or step-up authentication for sensitive operations, rate limiting, and comprehensive audit logs.

Alternative: For specialized real-time banking dashboards or secure event streams, WebSocket over TLS can be used where continuous bidirectional communication is genuinely required. For ordinary account and transaction APIs, HTTPS REST APIs remain simpler and more appropriate.

QUESTION 4: Third-Party API Integration Strategy

An application integrating payment gateways or map services should isolate third-party dependencies behind an integration layer. This prevents provider-specific details from spreading throughout the application and makes it easier to change providers later.

Proposed Integration Flow

Application → API Gateway → Integration Service → Third-Party API → Integration Service → Application

- Request handling: Validate the user's request, authorize the operation, create a correlation/request ID, and transform internal data into the provider's required format.
- Authentication: Store API keys, OAuth client secrets, or tokens in a secure secrets manager. Never hard-code credentials in source code or expose them to clients.
- Secure transport: Use HTTPS/TLS for all third-party API communication.
- Timeouts: Set reasonable connection and response timeouts so a slow provider does not block application resources indefinitely.
- Retries: Retry only transient failures using bounded exponential backoff and jitter. Avoid unsafe retries for non-idempotent payment operations unless idempotency keys are supported.
- Response processing: Validate HTTP status codes and response schemas, map provider errors to internal error codes, and sanitize returned data.
- Logging: Record request IDs, timing, status, and safe diagnostic information without storing secrets or unnecessary sensitive payment data.

Alternative Approach: Queue and Provider Adapter

For operations that do not need an immediate response, a message queue can decouple the application from the third-party provider. A provider adapter can also provide a common interface so that another payment or map provider can be substituted with minimal application changes.

- Queue-based processing absorbs traffic spikes and improves resilience when the third-party service is temporarily unavailable.
- Circuit breakers stop repeated calls to a failing provider and allow recovery without exhausting application resources.
- Caching can reduce repeated map or reference-data calls when the provider permits caching.
- A fallback provider can be used for critical services when business requirements and provider terms permit it.

QUESTION 5: Error Handling and Fault Tolerance

Frequent web-service errors should be handled at both the client and server sides. The goal is not only to prevent failures from becoming system-wide outages, but also to return clear and safe responses to users.

Proposed Mechanism

- Input validation: Validate required fields, data types, ranges, authentication, and authorization before business processing.

- Standard error responses: Return consistent HTTP status codes and a structured JSON error body containing a safe error code, message, and correlation ID.
- Centralized exception handling: Convert unexpected application exceptions into controlled responses rather than exposing stack traces.
- Timeouts: Prevent requests from waiting indefinitely for downstream services.
- Retry with exponential backoff: Automatically retry temporary network or service failures when the operation is safe to repeat.
- Circuit breaker: Temporarily stop calls to a failing dependency and return a controlled fallback response.
- Bulkhead isolation: Limit resources assigned to different workloads so one overloaded dependency does not consume all server resources.
- Health checks: Remove unhealthy instances from load balancing and restore them after successful health checks.
- Monitoring and alerting: Track error rate, latency, throughput, timeouts, and dependency failures.
- Idempotency: Use idempotency keys for operations such as payments where a retry must not create duplicate transactions.

Example Error Response

```
{
  "error": {
    "code": "SERVICE_UNAVAILABLE",
    "message": "The requested service is temporarily unavailable.",
    "correlationId": "REQ-2026-001245"
  }
}
```

Alternative Reliability Strategies

- Graceful degradation: Show cached or limited information when a non-critical dependency is unavailable.
- Asynchronous processing: Move long-running work to a queue and provide a job status mechanism.
- Fallback services: Use an approved secondary provider for critical external functionality.
- Load shedding and rate limiting: Reject excessive traffic early to protect healthy users and system resources.
- Disaster recovery: Maintain backups, replication, documented recovery procedures, and regular recovery testing.

These mechanisms provide a smoother user experience because users receive predictable messages instead of application crashes or indefinite waiting. Internally, correlation IDs, logs, metrics, and traces allow developers to identify the root cause and restore service quickly.

OVERALL SOLUTION SELECTION AND FEASIBILITY

Across the five problems, the recommended designs emphasize lightweight REST APIs for general mobile/web access, horizontally scalable application servers, HTTPS/TLS for secure communication,

isolated third-party integration services, and layered fault tolerance. The alternatives—SOAP, microservices, WebSocket over TLS, queue-based integrations, and fallback/circuit-breaker designs—remain valuable when their specific requirements justify their additional complexity.

The proposed architecture is feasible because it uses widely adopted web technologies and can be implemented incrementally. A basic deployment can start with a REST API, load balancer, cache, relational database, HTTPS, centralized error handling, and monitoring. As traffic and business complexity grow, additional application instances, read replicas, queues, microservices, and redundant providers can be introduced.

CODE OF CONDUCT

I, M.PAVITHRAN / 192411014 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M.PAVITHRAN

Name: M.PAVITHRAN

Register No.: 192411014

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Evidence in Submission
Problem Understanding	20%	Clearly defines problem, objectives, and requirements.	Each question identifies requirements and constraints.
Generation of Solutions	25%	Develops multiple feasible alternatives.	REST vs SOAP; monolith vs microservices; HTTPS vs WebSocket; queue/fallback options; fault-tolerance alternatives.
Evaluation of Alternatives	25%	Critically compares alternatives using suitable criteria.	Performance, scalability, security, reliability, complexity and feasibility are discussed.
Solution Selection	20%	Selects the best solution with strong rationale.	REST and HTTPS are selected for the stated primary scenarios with explicit justification.
Feasibility	10%	Applies relevant concepts practically.	Architecture uses load balancing, caching, TLS, queues, monitoring, retries and database scaling.